

Phym — Custom Collision Engine

Upright cylinders and axis-aligned boxes, resolved exactly in closed form, colliding one-way against the existing Unity level geometry.

Context

Rompe-Oros is a beat-'em-up greybox that today has **no movement or physics code at all**. `Assets/Scripts/P1layerTest.cs` only triggers sprite animations; nothing in the project has a `Rigidbody`, a `CharacterController`, or even a `FixedUpdate`. The `Phym` namespace exists as two data stubs — `Collider.cs` holds only a `Color`, and `CylinderCollider.cs` adds `radius / height` — plus the scene-view gizmo drawer.

The goal is a **simple but precise** collision engine that owns character movement, so motion is fully authored and predictable rather than negotiated with a rigidbody solver.

Locked decisions

Decision	Choice
Cylinder axis	Always upright (world +Y)
Boxes	Axis-aligned (AABB) only
Determinism	Not required — plain <code>float</code> / <code>Vector3</code>
Unity world	PhysX overlap queries for <i>broadphase only</i> ; all resolution in Phym math
Cylinder vs cylinder	Mass-weighted mutual push
Tunneling	Fixed sub-stepping (walking speeds + dashes)

Why these two shape choices matter

Both constraints make the math *exactly* solvable in closed form. This is the entire reason the engine can be simple and precise at the same time — there is no GJK, no EPA, and no iterative root-finding anywhere in the narrowphase.

- **Two upright cylinders:** their Minkowski sum *is another upright cylinder*, of radius `rA+rB` and height `hA+hB`. Exact penetration depth is two subtractions.
- **Upright cylinder + AABB:** the Minkowski sum is a vertical prism with a rounded-rectangle cross-section — constant in Y — so the exact minimum translation is `min(vertical exit, XZ exit)`.

Existing state worth knowing

- Test bed is `Assets/Scenes/SampleScene.unity`, the only scene, with no prefabs. `Player` (`r=0.32 h=1.3`) and `Dummy` (`r=0.25 h=0.55`) already carry `CylinderCollider`.
- Level geometry is **9 concave ProBuilder MeshColliders** — 3 floor slabs and 6 pillars. This is precisely why `Physics.ComputePenetration` and `Physics.ClosestPoint` are unusable: both reject non-convex meshes. `Physics.OverlapBox` works fine, so PhysX is used purely to *find candidates*.
- PhysX also has **no cylinder primitive**. Any PhysX-side resolution would approximate the character as a capsule — rounded feet, sliding off ledges early. Doing the resolution in Phym is what keeps it precise.

- Fixed timestep is already `0.02` (50 Hz) and the Input System is set to *Process Events In Fixed Update*, which lines up cleanly with ticking Phym in `FixedUpdate`.
- No assembly definitions exist; everything compiles into `Assembly-CSharp`.

Implementation status

Scaffolding exists in `Assets/Scripts/Phym/Runtime/`: `PhymBody`, `PhymCollider`, `PhymCylinder`, `PhymContact`, `PhymRegistry`, `PhymWorld`. Types were renamed to a `Phym*` prefix, which resolves the `UnityEngine.Collider` shadowing described later; the `.meta` GUIDs survived the rename, so the two scene components still bind, and the gizmo drawer is updated to the new type names.

`PhymRegistry` is complete and correct — deferred add/remove queues, adds processed before removes, and a `SubsystemRegistration` static reset. `PhymWorld.Discovery()` and `IntegrateForces()` are wired. The substep loop, narrowphase and solver are not yet written.

Fix before the first run

Bug	Consequence & fix
<code>PhymBody.Position</code> never initialised	Starts at <code>(0,0,0)</code> , so tick 1 drags every body to the origin. Seed <code>_position</code> , <code>_prevPosition</code> and <code>_lastWrittenPosition</code> from <code>transform.position</code> in <code>OnEnable</code> .
Nothing calls <code>PhymRegistry.Register</code>	<code>_bodies</code> is permanently empty and the tick loops zero times. Add <code>OnEnable</code> / <code>OnDisable</code> to <code>PhymBody</code> . Neither <code>Player</code> nor <code>Dummy</code> carries a <code>PhymBody</code> component yet.
<code>TELEPORT_EPSILON_SQ = 0.1f</code>	It is a <i>squared</i> threshold, so the real distance is ~32 cm and any smaller external move is ignored. The transform is never interpolated, so this should be noise-sized: <code>1e-8f</code> .
<code>IsGrounded</code> never reset	<code>Discovery()</code> clears <code>GroundNormal</code> but not the flag, so once grounded a body stays grounded forever.
<code>Gravity</code> defaults to <code>0</code>	Serialized field, so nothing falls until set in the Inspector. Default it to <code>-9.81f</code> in the declaration.
<code>WorldBounds</code> is not <code>virtual</code>	<code>PhymCylinder</code> cannot override it, so every collider returns the 1 cm placeholder and the broadphase rejects every pair. Also the natural home for the <code>lossyScale</code> bake, which still lives only in the gizmo drawer.

Minor: a stray `using UnityEngine.UIElements;` in `PhymWorld`; no static/immovable skip in `IntegrateForces`; and `struct Contact` in `PhymContact.cs` breaks the `Phym*` convention.

Velocity layers

Characters need *authored* motion, not force integration. `AddForce` is not imprecise — float maths is deterministic — but it is **indirect**: you specify a force, and mass, accumulation and `dt` decide the outcome. A character controller wants directness, so a single `Velocity` field is replaced by a fixed set of layers that sum, in deterministic order, to the velocity used for integration and solving.

The pattern is well established: Source keeps `basevelocity` separate from `velocity`, and Unreal's `CharacterMovementComponent` holds root motion and the moving-base contribution apart from the character's own velocity. It buys three things here — the **Move** layer is *set* rather than accumulated, so accel/decel curves are entirely yours; knockback decays on its own schedule without stomping player control, and input does not erase knockback; and conveyors or moving platforms contribute additively without polluting the character's own velocity.

```
public enum VelocityLayer
{
    Move,           // input-driven; set directly each tick
    Gravity,        // accumulates
    Knockback,      // impulse-driven; decays
    External,       // conveyors, platforms, wind
    Resolution,     // written ONLY by the solver
    Count
}

// NOT [SerializeField] -- see the serialisation trap below.
Vector3[] _layers = new Vector3[(int)VelocityLayer.Count];

public Vector3 this[VelocityLayer l]
{
    get => _layers[(int)l];
    set => _layers[(int)l] = value;
}

// Read-only by design: callers set a layer, never the total. Summed in fixed
// enum order, because float addition is not associative and a Dictionary's
// iteration order would make the total vary run to run.
public Vector3 Velocity
{
    get
    {
        Vector3 sum = Vector3.zero;
        for (int i = 0; i < _layers.Length; i++)
            sum += _layers[i];
        return sum;
    }
}
```

Never `[SerializeField]` the layer array. Unity serialises array *length*, so adding an enum entry later leaves every existing component holding a short array and throwing `IndexOutOfRangeException` on load. Velocity is runtime state, not authored data. For Inspector visibility while debugging, an editor-only drawer under `if (Application.isPlaying)` gives more than serialisation would — named rows plus the summed total.

The solver never touches the gameplay layers

Collision response accumulates into `Resolution` alone, leaving Move, Gravity, Knockback and External exactly as gameplay authored them. `Resolution` is **per-tick scratch**, cleared in `Discovery()` beside `IsGrounded`. That is what keeps it from going stale, and it needs no decay rule: pressing into a wall re-derives the same correction every tick, and stepping away simply drops it.

It composes with the iterative solver for free. Because `Velocity` is a sum that *includes* `Resolution`, each iteration's `vn` already reflects the corrections written by previous iterations — an already-cancelled velocity yields `vn == 0` and no further write. The same argument holds across substeps, which is why clearing once per tick is correct rather than once per substep.

The companion rule this requires

Under a single `Velocity` field the solver cancelled velocity directly, which happened to reset gravity as a side effect. It cannot any more — so standing still on a floor, the `Gravity` layer accumulates $-9.81 * t$ indefinitely. Nothing looks wrong, because the sum stays at zero while `Resolution` cancels it. Then the character walks off a ledge after a minute and drops at 588 m/s.

Purity has to be paid for with an explicit rule, post-solve, once grounding has been classified:

```
void PostSolve()
{
    for (int i = 0; i < _bodies.Count; i++)
    {
        PhymBody body = _bodies[i];
        if (!body.IsGrounded) continue;

        // The solver never touches Gravity now, so it would grow without bound
        // while standing still. Grounding is the explicit rule that resets it.
        Vector3 g = body[VelocityLayer.Gravity];
        if (g.y < 0f)
        {
            g.y = 0f;
            body[VelocityLayer.Gravity] = g;
        }
    }
}
```

This is a feature rather than a workaround: the reset becomes a visible gameplay rule you control, instead of an emergent side effect of the solver. `Velocity` losing its setter is likewise deliberate — external code sets `body[VelocityLayer.Move]`, and a `ResetVelocity()` that zeroes every layer covers teleports and respawns.

The tick pipeline

One `PhymWorld.FixedUpdate` drives everything. Individual colliders have no update of their own, so ordering is explicit rather than emergent.

```
// PhymWorld.cs
void Tick(float dt)
{
    RefreshBodies();           // reconcile with Unity transforms, bake world shapes
    IntegrateForces(dt);        // gravity, desired velocity

    // No swept tests. Instead, guarantee no body moves further than half
    // the smallest radius in one substep -- which makes tunneling impossible
    // for plain discrete overlap tests, for about fifteen lines of code.
    float maxMotion = 0f;
    for (int i = 0; i < _dynamic.Count; i++)
        maxMotion = Mathf.Max(maxMotion, _dynamic[i].Velocity.magnitude * dt);

    int substeps = Mathf.Clamp(Mathf.CeilToInt(maxMotion / MaxStepDistance), 1, MaxSubsteps);
    float h = dt / substeps;

    for (int s = 0; s < substeps; s++)
    {
        for (int i = 0; i < _dynamic.Count; i++)
            _dynamic[i].Position += _dynamic[i].Velocity * h;

        _contacts.Clear();
        BuildContacts(_contacts);           // detection
        _solver.Solve(_contacts, SolverIterations); // resolution
    }

    ClassifyGrounding(); // contact normals -> grounded / slope
    PostSolve();         // zero the Gravity layer while grounded
    WriteBackTransforms(); // raw physics position, never smoothed
}
```

The physics transform is never interpolated. `WriteBackTransforms()` assigns the exact solver position to `transform.position`, once per tick, and nothing in the engine ever writes a smoothed value there. The root transform is the physics state — what you read in the Inspector is what the solver computed. Any visual smoothing is a separate, optional concern living on a child node; see *Render interpolation* below.

What tunneling is, and why sub-stepping fixes it

Tunneling is when a fast-moving object passes clean through something it should have hit, because collision detection only ever samples discrete positions in time. At 50 Hz each tick is 0.02 s, and discrete detection asks exactly one question per tick — “are these shapes overlapping right now?” It never asks what happened in between.

Take the Player (`r=0.32`) dashing at 60 m/s — 1.2 units per tick — at a pillar spanning `z = -4.25 .. -3.75`:

tick N	z = -3.4	pillar spans -4.25 .. -3.75	-> no overlap
tick N+1	z = -4.6	pillar spans -4.25 .. -3.75	-> no overlap

Neither sampled position overlaps, so no contact is ever generated, and the character is now standing on the far side of a solid wall. The rule of thumb: tunneling becomes possible the moment an object moves further than its own thickness in a single tick. *Small and fast* is the dangerous combination, which is why bullets are the textbook example.

Sub-stepping fixes it by shrinking the sampling interval until that can't happen: the 1.2-unit move becomes 8 steps of 0.15, and one of those steps lands inside the pillar. It costs nothing but a loop, and it reuses the exact same overlap tests already written — no separate swept code path. `MaxSubsteps` is the safety valve: past some absurd speed you clamp and accept a tunnel, rather than running 500 substeps and hitching the frame.

The alternative is **true continuous detection** — solving for the exact time of impact along the whole motion segment rather than sampling at all. It never misses at any speed, but it is real math per shape pair with its own edge cases. That is what actual projectiles would need, and it is why the speed question was worth settling up front.

RefreshBodies — reconciling with Unity's transforms

This is the handshake between Unity's transform world and Phym's internal state. It answers one question at the top of every tick: *did anything move or change behind my back since I last ran?*

The core idea is an ownership split. **During** the tick Phym owns position — the solver reads and writes `body.Position`, a plain `Vector3` field, and the transform is not touched at all. **Between** ticks the transform is fair game for everyone else: spawning code, a cutscene teleport, a designer dragging the object in the Inspector during play mode. `RefreshBodies()` is the single point where that outside world gets pulled back in.

```
void RefreshBodies()
{
    for (int i = 0; i < _all.Count; i++)
    {
        PhymBody body = _all[i];

        // 1. Teleport detection. Compare the transform against what Phym
        //    itself last wrote. If they differ, somebody else moved this
        //    object and their intent wins -- along with dropping stale
        //    ground state, since the body is somewhere entirely new.
        Vector3 authored = body.Transform.position;
        if ((authored - body.LastWrittenPosition).sqrMagnitude > TeleportEpsilonSq)
        {
            body.Position = authored;
            body.IsGrounded = false;
        }

        // 2. Snapshot for render interpolation: the visual lerp over the next
        //    0.02s starts here and ends wherever this tick leaves us.
        body.PrevPosition = body.Position;

        // 3. Bake authored shape -> solver shape. radius/height are authored in
        //    local space; lossyScale converts them. This walks the parent
        //    hierarchy, so it is the expensive part -- and the reason this runs
        //    once per tick rather than inside the substep loop.
        body.Collider.RefreshWorldShape(); // -> WorldRadius, WorldHalfHeight

        // 4. Clear per-tick outputs so last frame's state cannot leak forward.
        body.GroundNormal = Vector3.zero;
        body.IsGrounded = false;
    }
}
```

Why detect teleports instead of always assigning from the transform?

Two reasons, and the second is why the never-interpolate rule pays off.

First, **precision**. If the body is parented, reading `transform.position` back every tick round-trips the value through the parent's matrix, and the error accumulates. Keeping `body.Position` authoritative and only overriding it on a real external write avoids that entirely.

Second, **the epsilon can be tiny**. Because the root transform is never smoothed, between ticks it holds exactly what `WriteBackTransforms()` put there. Any deviation beyond float round-trip noise is therefore, unambiguously, somebody else's write:

```
// Noise-sized, not behaviour-sized. A 1 cm teleport is still detected.
const float TeleportEpsilonSq = 1e-8f;
```

Had the root been interpolated, this threshold would have to exceed the interpolation swing — potentially tens of centimetres at speed — and every teleport smaller than that would be silently swallowed. Worse, reading a smoothed transform back into the simulation feeds the interpolator's output into its own input, and the body creeps or oscillates on its own. Keeping the root raw removes that whole class of bug rather than tuning around it.

Bounds are derived, not cached here

It is tempting to compute each body's world AABB inside `RefreshBodies()` alongside the shape bake. That would be a bug: `RefreshBodies()` runs **once per tick**, but positions advance **inside the substep loop**, so a cached AABB is stale from substep 2 onward and the broadphase would cull against the previous substep's bounds. Fast-moving bodies would miss contacts intermittently — the worst kind of bug to chase.

Split it so the expensive part is cached and the cheap part is derived on demand:

```
// Cached once per tick by RefreshWorldShape() -- lossyScale is the costly bit,
// and scale cannot change mid-tick anyway.
public float WorldRadius, WorldHalfHeight;

// Derived per substep from the live Position -- pure arithmetic, effectively free.
public Bounds WorldBounds => new Bounds(
    Position,
    new Vector3(WorldRadius * 2f, WorldHalfHeight * 2f, WorldRadius * 2f));
```

Detection — building this frame's contact set

Contacts are appended to a single `List<Contact>` reused across frames, so the whole pipeline is allocation-free after warm-up. The list is then sorted by depth descending, with body IDs as a stable tiebreak so resolution order does not flicker frame to frame.

```
// The contact record. B is null for Unity-world contacts, which are
// then treated as infinite mass. Normal always points from B toward A.
public struct Contact
{
    public PhymBody A;
    public PhymBody B;
    public Vector3 Normal;
    public float Depth;
    public Vector3 Point; // debug visualisation only
}
```

```

void BuildContacts(List<Contact> into)
{
    for (int i = 0; i < _dynamic.Count; i++)
    {
        PhymBody body = _dynamic[i];
        Bounds aabb = body.WorldBounds;
        aabb.Expand(ContactOffset * 2f);

        // --- Phym vs Phym -----
        // Brute-force AABB culling. With tens of bodies this genuinely beats
        // a spatial hash; it sits behind an interface so a grid can drop in.
        for (int j = i + 1; j < _all.Count; j++)
            if (aabb.Intersects(_all[j].WorldBounds))
                Narrowphase.Generate(body, _all[j], into);

        // --- Phym vs Unity (one-way) -----
        // PhysX finds the candidates; Phym does every bit of the resolution.
        int hits = Physics.OverlapBoxNonAlloc(
            aabb.center, aabb.extents, _probeBuffer,
            Quaternion.identity, BlockingMask, QueryTriggerInteraction.Ignore);

        for (int h = 0; h < hits; h++)
            UnityWorldProbe.Generate(body, _probeBuffer[h], into);
    }
}

```

UnityWorldProbe dispatches per Unity collider type: a MeshCollider yields cached world-space triangles (concave or not — it never matters, because Phym does the math); a UnityEngine.BoxCollider yields its 12 triangles, which is *exact* since a box surface genuinely is 12 triangles; spheres and capsules get analytic tests.

Narrowphase math

Everything below lives in `Narrowphase.cs` as **pure static functions** — no `MonoBehaviour`, no world state. That makes them directly unit-testable, which matters because these are the errors that are miserable to debug in-scene.

Cylinder vs cylinder

The exact case. Because the Minkowski sum of two upright cylinders is itself an upright cylinder, the shortest exit is simply the smaller of "out through the side wall" and "out through a cap".

```
public static bool CylinderVsCylinder(in Cylinder a, in Cylinder b, out Contact contact)
{
    contact = default;

    Vector3 d = a.Center - b.Center;

    // Radial test in XZ. Both penetration depths below are exact -- the
    // configuration-space obstacle is a cylinder of radius rA+rB and
    // height hA+hB, so there is nothing to iterate toward.
    float radiusSum = a.Radius + b.Radius;
    float distXZSq = d.x * d.x + d.z * d.z;
    if (distXZSq >= radiusSum * radiusSum)
        return false;

    float penY = (a.HalfHeight + b.HalfHeight) - Mathf.Abs(d.y);
    if (penY <= 0f)
        return false;

    float distXZ = Mathf.Sqrt(distXZSq);
    float penXZ = radiusSum - distXZ;

    // Minimum translation vector = whichever exit is shorter.
    if (penY < penXZ)
    {
        // Cap contact: straight up or straight down. This is what lets one
        // character stand on another's head instead of squeezing sideways.
        contact.Normal = new Vector3(0f, d.y >= 0f ? 1f : -1f, 0f);
        contact.Depth = penY;
    }
    else if (distXZ > Epsilon)
    {
        // Side contact: push along the horizontal centre-to-centre axis.
        contact.Normal = new Vector3(d.x / distXZ, 0f, d.z / distXZ);
        contact.Depth = penXZ;
    }
    else
    {
        // Degenerate: centres exactly stacked. Pick a stable arbitrary
        // direction so co-spawned characters separate instead of NaN-ing.
        contact.Normal = Vector3.right;
        contact.Depth = penXZ;
    }

    contact.A = a.Body;
    contact.B = b.Body;
    contact.Point = a.Center - contact.Normal *
        (contact.Normal.y != 0f ? a.HalfHeight : a.Radius);
    return true;
}
```

Cylinder vs AABB

Same principle, one dimension harder: the vertical axis is a plain interval test, and the horizontal plane reduces to **circle versus rectangle**. The interesting branch is when the cylinder's centre lies *inside* the box footprint.

```

public static bool CylinderVsAabb(in Cylinder c, in Aabb box, out Contact contact)
{
    contact = default;

    // ---- Vertical axis: plain interval overlap -----
    float dy = c.Center.y - box.Center.y;
    float penY = (box.HalfExtents.y + c.HalfHeight) - Mathf.Abs(dy);
    if (penY <= 0f)
        return false;

    // ---- Horizontal plane: circle vs rectangle -----
    float dx = c.Center.x - box.Center.x;
    float dz = c.Center.z - box.Center.z;
    float hx = box.HalfExtents.x;
    float hz = box.HalfExtents.z;

    float penXZ;
    Vector3 normalXZ;

    if (Mathf.Abs(dx) <= hx && Mathf.Abs(dz) <= hz)
    {
        // Centre is INSIDE the footprint -- standing on top of the box, or
        // fully embedded in it. Distance is now measured outward to the
        // nearest face, so horizontal penetration is the radius PLUS that
        // inset. That large value is exactly why penY wins here, and why a
        // character on a crate gets pushed UP rather than shoved sideways.
        float insetX = hx - Mathf.Abs(dx);
        float insetZ = hz - Mathf.Abs(dz);

        if (insetX < insetZ)
        {
            penXZ = c.Radius + insetX;
            normalXZ = new Vector3(Mathf.Sign(dx), 0f, 0f);
        }
        else
        {
            penXZ = c.Radius + insetZ;
            normalXZ = new Vector3(0f, 0f, Mathf.Sign(dz));
        }
    }
    else
    {
        // Centre is OUTSIDE: clamp it onto the rectangle to get the closest
        // point, and it collapses to circle-vs-point. Two lines handle the
        // face case and the corner case identically -- no special-casing.
        float qx = Mathf.Clamp(dx, -hx, hx);
        float qz = Mathf.Clamp(dz, -hz, hz);

        float ox = dx - qx, oz = dz - qz;
        float distSq = ox * ox + oz * oz;
        if (distSq >= c.Radius * c.Radius)
            return false;

        float dist = Mathf.Sqrt(distSq);
        penXZ = c.Radius - dist;
        normalXZ = dist > Epsilon
            ? new Vector3(ox / dist, 0f, oz / dist)
            : Vector3.right;
    }

    // ---- Minimum translation vector -----
    if (penY < penXZ)
    {
        contact.Normal = new Vector3(0f, dy >= 0f ? 1f : -1f, 0f);
        contact.Depth = penY;
    }
}

```

```
else
{
    contact.Normal = normalXZ;
    contact.Depth  = penXZ;
}

contact.Point = c.Center - contact.Normal * c.Radius;
return true;
}
```

Sign discipline. Every test returns a normal pointing *from B toward A*, and a strictly positive depth. Getting this consistent across all four pair types is worth a dedicated unit test — `Test(a,b).Normal == -Test(b,a).Normal` with equal depth — because an inverted normal produces a body that gets sucked *into* geometry rather than pushed out, and that reads as a mysterious teleport rather than an obvious sign error.

Cylinder vs triangle

The only non-trivial test, and the one that meets the Unity world. Splitting it by triangle orientation is both faster and far more robust than one general path.

```
public static bool CylinderVsTriangle(in Cylinder c, in Triangle t, out Contact contact)
{
    // Floors and ceilings resolve along the face normal. Walls resolve as a
    // 2D circle-vs-segment problem inside the cylinder's Y slab.
    //
    // The split is a large robustness win. The classic triangle-soup bug --
    // a character snagging on the invisible seam between two coplanar quads
    // -- comes from edge-region normals, and the wall path has no edge
    // regions at all: it only ever sees vertical line segments.
    return Mathf.Abs(t.Normal.y) > WallThreshold // 0.5 == 60 degrees
        ? CylinderVsFloorTriangle(c, t, out contact)
        : CylinderVsWallTriangle(c, t, out contact);
}
```

Wall path — clip to the Y slab, then solve in 2D

```
static bool CylinderVsWallTriangle(in Cylinder c, in Triangle t, out Contact contact)
{
    contact = default;

    // Clip the triangle to the horizontal slab the cylinder occupies. What
    // survives is a convex polygon of 0..5 vertices; anything above or below
    // the character simply cannot touch it, and drops out here for free.
    int n = ClipToYSlab(t, c.Center.y - c.HalfHeight,
                       c.Center.y + c.HalfHeight, _clipBuffer);

    if (n < 2)
        return false;

    // Project to XZ and find the deepest circle-vs-segment overlap. Because
    // the cylinder cross-section is constant in Y, this 2D answer IS the
    // exact 3D answer -- no approximation is being made here.
    Vector2 centre = new Vector2(c.Center.x, c.Center.z);
    float bestDepth = 0f;
    Vector2 bestNormal = default;

    for (int i = 0; i < n; i++)
    {
        Vector2 p0 = Flatten(_clipBuffer[i]);
        Vector2 p1 = Flatten(_clipBuffer[(i + 1) % n]);

        if (CircleVsSegment(centre, c.Radius, p0, p1, out Vector2 nrm, out float depth)
            && depth > bestDepth)
        {
            bestDepth = depth;
            bestNormal = nrm;
        }
    }

    if (bestDepth <= 0f)
        return false;

    contact.Normal = new Vector3(bestNormal.x, 0f, bestNormal.y);
    contact.Depth = bestDepth;
    contact.Point = c.Center - contact.Normal * c.Radius;
    return true;
}

static bool CircleVsSegment(Vector2 c, float r, Vector2 p0, Vector2 p1,
                           out Vector2 normal, out float depth)
{
    Vector2 e = p1 - p0;
    float eLenSq = e.sqrMagnitude;

    // Clamp01 is what turns this from an infinite-line test into a segment
    // test, and it is also what handles the corner case: when t clamps to
    // 0 or 1 the closest feature is a vertex, and the normal comes out
    // radial from that vertex automatically.
    float t = eLenSq > Epsilon ? Mathf.Clamp01(Vector2.Dot(c - p0, e) / eLenSq) : 0f;
    Vector2 offset = c - (p0 + e * t);

    float distSq = offset.sqrMagnitude;
    if (distSq >= r * r)
    {
        normal = default;
        depth = 0f;
        return false;
    }

    float dist = Mathf.Sqrt(distSq);
    normal = dist > Epsilon ? offset / dist : new Vector2(-e.y, e.x).normalized;
    depth = r - dist;
}
```

```
    return true;  
}
```

Internal edges are the classic failure mode of triangle-soup collision. Mitigations, cheapest first: **(a)** the wall/floor split above, which removes the problem for all vertical geometry; **(b)** after generating one body's triangle contacts, cluster normals within $\sim 5^\circ$ and keep only the deepest per cluster; **(c)** reject edge- and vertex-region contacts whose normal points into the front side of a neighbouring triangle. Start at (a), and only add (b) or (c) if snagging actually shows up in testing.

Resolution — the solver

Iterative relaxation over the contact list, 4–8 passes. Static Phym bodies and the Unity world both fall out naturally as `inverseMass == 0`; an "immovable" boss character is the same flag, not a special case.

```
public void Solve(List<Contact> contacts, int iterations)
{
    for (int iter = 0; iter < iterations; iter++)
    {
        for (int i = 0; i < contacts.Count; i++)
        {
            Contact c = contacts[i];

            // Re-evaluate against CURRENT positions rather than trusting the
            // depth captured during detection. This single detail decides
            // whether a body wedged in a corner settles or vibrates: without
            // it, each constraint keeps correcting an overlap that an earlier
            // constraint in the same pass already removed, and they fight.
            if (!Narrowphase.Refresh(ref c))
                continue;

            PhymBody a = c.A;
            PhymBody b = c.B; // null == static Unity world

            float wA = a.InverseMass;
            float wB = b != null ? b.InverseMass : 0f;
            float wSum = wA + wB;
            if (wSum <= 0f)
                continue; // two statics -- nothing to do

            // Statics get the full correction so the character never sinks into
            // the floor. Dynamic pairs get a softer bias so a crowd compresses
            // and relaxes instead of exploding apart on the first frame.
            float bias = b == null ? StaticBias : DynamicBias; // 1.0 / 0.8

            // Positional correction, split by inverse mass. A heavy body barely
            // moves; an infinite-mass body does not move at all.
            Vector3 correction = c.Normal * (c.Depth * bias);
            a.Position += correction * (wA / wSum);
            if (b != null)
                b.Position -= correction * (wB / wSum);

            // Cancel ONLY the velocity driving into the surface. The tangential
            // component survives untouched -- and that is precisely what makes
            // the character slide along a wall instead of sticking to it.
            //
            // The correction lands in the Resolution layer, never in Move, Gravity
            // or Knockback: gameplay layers stay exactly as authored. Velocity is a
            // sum that INCLUDES Resolution, so the next iteration's vn already sees
            // this write, and an already-cancelled contact yields vn == 0.
            Vector3 vRel = a.Velocity - (b != null ? b.Velocity : Vector3.zero);
            float vn = Vector3.Dot(vRel, c.Normal);
            if (vn < 0f)
            {
                Vector3 impulse = c.Normal * -vn;
                a[VelocityLayer.Resolution] += impulse * (wA / wSum);
                if (b != null)
                    b[VelocityLayer.Resolution] -= impulse * (wB / wSum);
            }
        }
    }
}
```

Grounding, from the same contact set

No extra raycasts. The contacts already generated this tick carry everything needed to classify ground state, which keeps the answer consistent with what the solver actually did.

```
void ClassifyGrounding()
{
    float minNormalY = Mathf.Cos(MaxSlopeAngle * Mathf.Deg2Rad); // e.g. 50 degrees

    for (int i = 0; i < _contacts.Count; i++)
    {
        Contact c = _contacts[i];

        // Normal points from B toward A, so a normal with a strong +Y
        // component means the surface is holding A up: that is ground.
        if (c.Normal.y < minNormalY)
            continue;

        PhymBody body = c.A;
        if (c.Normal.y > body.GroundNormal.y) // keep the flattest contact
        {
            body.IsGrounded = true;
            body.GroundNormal = c.Normal;
        }
    }
}
```

Files

Nothing existing is thrown away. `Collider.cs` and `CylinderCollider.cs` grow; the gizmo drawer stays as it is.

```
Assets/Scripts/Phym/
├── Phym.asmdef                                NEW
├── Runtime/
│   ├── PhymWorld.cs                          NEW    registry + FixedUpdate tick driver
│   ├── PhymBody.cs                          NEW    velocity, mass, prev/lastWritten position
│   ├── Collider.cs                          GROW    WorldBounds, OnEnable/OnDisable registration
│   ├── CylinderCollider.cs                  GROW    WorldRadius / WorldHalfHeight / WorldCenter
│   ├── BoxCollider.cs                      NEW    centre offset + size -> world AABB
│   ├── Shapes.cs                          NEW    Cylinder / Aabb / Triangle structs
│   ├── Contact.cs                         NEW    contact record
│   ├── Narrowphase.cs                     NEW    all pair tests, pure static
│   ├── Broadphase.cs                     NEW    AABB pair culling
│   ├── UnityWorldProbe.cs                 NEW    OverlapBoxNonAlloc -> contacts
│   ├── MeshTriangleCache.cs              NEW    MeshCollider -> cached world triangles
│   ├── Solver.cs                         NEW    relaxation + velocity projection
│   └── PhymCharacter.cs                   NEW    gravity, grounding, slope, step-up
├── Editor/
│   ├── Phym.Editor.asmdef                 NEW
│   ├── CylinderColliderGizmoDrawer.cs     unchanged
│   ├── BoxColliderGizmoDrawer.cs         NEW    reuses existing gizmo patterns
│   └── PhymDebugOverlay.cs               NEW    live contacts, normals, depths, ground state
Assets/Scripts/Visual/
├── PhymVisualInterpolator.cs              NEW    optional, opt-in, outside the engine
Assets/Tests/Phym/
├── Phym.Tests.asmdef                     NEW
└── NarrowphaseTests.cs                   NEW
```

Reuse

`CylinderColliderGizmoDrawer.cs` already contains the world-scale convention — `lateral = max(|lossy.x|, |lossy.z|)`, `height * |lossy.y|` — and a reusable 32-segment unit-cylinder mesh builder. **Extract that convention into `CylinderCollider.WorldRadius / WorldHalfHeight` and have the gizmo call those.** Right now it lives only in editor code, which becomes a latent bug the moment the solver computes its own: the drawn shape and the solved shape would silently disagree.

Assembly definitions — create these first

An **assembly definition** (`.asmdef`) tells Unity to compile a folder into its own .NET assembly instead of the project-wide default. The project currently has none, so every script — `PllayerTest.cs`, `SpriteAnimator.cs`, `Phym/Collider.cs` — lands in the auto-generated `Assembly-CSharp`. Dropping `Phym.asmdef` into `Assets/Scripts/Phym/` makes that folder compile to `Phym.dll`:

```
{
  "name": "Phym",
  "rootNamespace": "Phym",
  "references": [],
  "includePlatforms": [],
  "allowUnsafeCode": false
}
```

What that buys:

- **Faster iteration.** Unity recompiles only changed assemblies. Editing a sprite script stops rebuilding the physics engine, and vice versa.
- **Enforced dependency direction.** An `asmdef` only sees what it lists in `references`. With `references: []`, `Phym` *cannot* call into gameplay code — the compiler rejects it. Gameplay references `Phym`, never the reverse.
- **Per-assembly settings.** `allowUnsafeCode`, `Burst`, platform inclusion — scoped to this assembly rather than flipped globally in Player Settings, where it is currently off.

Why this is step 1 rather than a nice-to-have: unit tests. Unity test assemblies are themselves `asmdefs`, and an `asmdef` *cannot* reference the predefined `Assembly-CSharp` — it is invisible to them. So while `Phym` lives there, no `EditMode` test can call `Narrowphase.CylinderVsCylinder` at all. Timing is the other argument: two placed components is a trivially small blast radius, two hundred is not. Components serialize by `MonoScript` GUID, which does not change when the assembly does, so the two `CylinderColliders` in `SampleScene` survive the move intact.

Authoring them: do not hand-write the JSON. Right-click the folder → *Create* → *Scripting* → *Assembly Definition*, then configure in the Inspector, which has a proper UI and rewrites the file for you. Modern Unity stores references as GUIDs (`"GUID:6055be8e..."`) rather than names — unreadable, but rename-proof, and the Inspector handles it. For the test assembly specifically, use *Create* → *Testing* → *EditMode Test Assembly Folder*: it auto-fills the `nunit` reference, the test-framework reference, `"includePlatforms": ["Editor"]` and the `TestAssemblies` flag. Getting that combination wrong by hand produces no error — Test Runner simply shows no tests. `Phym.Editor.asmdef` likewise needs `"includePlatforms": ["Editor"]` plus a reference to `Phym`, or the gizmo drawer will not compile in a build.

Naming collision

`Phym.Collider` already shadows `UnityEngine.Collider`, and `Phym.BoxCollider` will shadow `UnityEngine.BoxCollider`. Only `UnityWorldProbe.cs` and `MeshTriangleCache.cs` need both, so alias at the top of those two files:

```
using UCollider    = UnityEngine.Collider;
using UBoxCollider = UnityEngine.BoxCollider;
```

Build order

1. **Asmdefs + shape foundation.** Three `asmdefs`. `PhymBody`, `Shapes`, `Contact`. Grow `Collider` with `WorldBounds` and registration; grow `CylinderCollider` with world-scale accessors and repoint the gizmo drawer at them. Add `BoxCollider` and its gizmo drawer.
2. **PhymWorld + tick skeleton.** `[DefaultExecutionOrder]` so it runs after input. Set `Physics.simulationMode = SimulationMode.Script` in `Awake` and never call `Simulate` — queries keep working, and the project stops paying for a solver nothing uses.
3. **Narrowphase: cylinder-cylinder and cylinder-AABB.** Pure functions plus unit tests. Get the precise core exact before anything is wired up.
4. **Broadphase + Solver.** Brute-force AABB culling behind an interface, then the relaxation loop.
5. **Phym-vs-Phym is playable.** Add temporary WASD movement alongside `PlayerTest.cs`; verify the Player pushes the Dummy in `SampleScene`.
6. **MeshTriangleCache + UnityWorldProbe + cylinder-vs-triangle.** The 9 ProBuilder colliders come alive. Cache world-space triangles once per static collider, with a per-triangle AABB for culling.
7. **PhymCharacter.** Gravity, ground snapping, slope projection, step-up.
8. **Render interpolation — optional, and outside the engine.** See below. The physics root is left raw regardless; this step only decides whether a visual child smooths on top of it.

Render interpolation

The engine never smooths the physics transform. If visual smoothing is wanted, it goes on a **visual child**, as an opt-in component that reads `PrevPosition / Position` and is not referenced by the solver at all — remove it and the simulation is bit-identical.

The scene hierarchy already suits this. `Player` carries the `CylinderCollider` and two visual children, `Shadow` and `Animation`. Wrapping both in a single `Visual` node gives one place to attach the component:

```
Player      [PhymCharacter, CylinderCollider]  <- raw physics position
└─ Visual   [PhymVisualInterpolator]          <- the only smoothed node
    └─ Shadow [SpriteRenderer]
    └─ Animation [SpriteAnimator]
```

```
// OPTIONAL. Deliberately not part of the engine -- nothing in Phym reads this,
// and deleting it changes rendering only, never simulation.
[DefaultExecutionOrder(1000)]
public class PhymVisualInterpolator : MonoBehaviour
{
    public PhymBody Body;

    void LateUpdate()
    {
        // How far through the current fixed step this frame is drawing.
        float alpha = Mathf.Clamp01((Time.time - Time.fixedTime) / Time.fixedDeltaTime);

        Vector3 smoothed = Vector3.Lerp(Body.PrevPosition, Body.Position, alpha);

        // The parent already sits at the raw physics position, so express the
        // smoothed world position as a local offset from it. No reparenting, and
        // the visual still inherits the root's rotation and scale.
        transform.localPosition =
            transform.parent.InverseTransformVector(smoothed - transform.parent.position);
    }
}
```

Whether to use it at all is a taste call, and this project is an unusually weak case for it. The camera rig is static — no follow script — and stutter is far more visible when the camera itself moves in 50 Hz steps. The character sprite is already quantised to 12 fps by `SpriteAnimator`. And interpolation renders one tick in the past, adding up to 20 ms of input latency, which a beat-'em-up feels more than most genres. Ship without it, and add it only if 50 Hz motion actually reads as choppy on a high-refresh display.

Steps 1–5 form a self-contained milestone: cylinders colliding with cylinders and boxes, fully working, before any Unity-world complexity enters.

Verification

Unit tests

`Assets/Tests/Phym/NarrowphaseTests.cs`, EditMode, via Window → General → Test Runner. The narrowphase being pure functions makes this cheap, and it catches exactly the errors that are painful to diagnose in-scene.

- Two cylinders with a gap → no contact; overlapping → depth equals the hand-computed `(rA+rB) - distXZ`.
- Two cylinders whose vertical overlap is smaller than their radial overlap → normal is `±Y`, not radial. *Least-penetration axis selection is the single most common place to get the `min` or the sign backwards.*
- Cylinder centred inside a box footprint, resting on top → normal `+Y`, depth `= (halfY + halfHeight) - |dy|`.
- Cylinder against a box *corner* → normal along the diagonal, not along a face axis.
- Coincident cylinder centres → still a valid unit normal, no `NaN`.
- Symmetry, for every pair type: `Test(a,b).Normal == -Test(b,a).Normal`, equal depth.

In-scene smoke tests

In `SampleScene`, with `PhymDebugOverlay` drawing live contact points, normals and depths.

1. Walk the Player into the Dummy — both slide apart, the lighter one further. Set the Dummy to infinite mass and confirm only the Player is displaced.
2. Walk into a pillar and strafe — smooth slide around it, no catching, no vibration while pressed into it.
3. Walk along a floor-slab seam — must not snag. This is the internal-edge check; if it catches, add the normal-clustering mitigation.
4. Walk off a slab edge — should fall cleanly at the edge. Because this is a true cylinder rather than a capsule, it must *not* slide off early.
5. Dash at high speed into a pillar — must not tunnel. Temporarily set `MaxSubsteps = 1` to confirm it *does* tunnel, then restore it: that proves sub-stepping is actually doing its job rather than being incidentally masked.
6. Stand still on a floor for 30 seconds and watch `transform.position.y`. It must not drift or oscillate; drift means the bias and skin values need tuning.
7. Spawn 10 cylinders overlapping at one point — they should push apart and settle without endless jitter.

Gotcha: `MeshCollider.sharedMesh` triangle reads require a readable mesh. Scene-embedded ProBuilder meshes are readable, but any imported model collider needs *Read/Write Enabled*. `MeshTriangleCache` should check `mesh.isReadable` and log a clear one-time error naming the offending `GameObject`, rather than throwing from inside the physics tick.